C64uRemote for M5Stack Core

Technical Documentation

Version 1.0

Contents

Overview

C64uRemote is firmware for the M5Stack Core (ESP32) that remotely controls the Commodore 64 Ultimate and Ultimate64 Elite-II via its ReST API. It also supports an RFID2 reader (NXP WS1850S) and a microSD card to launch programs from NFC cards and to manage cards.

The firmware is implemented as a single translation unit (`src/main.cpp`, about 4200 lines) and uses the Arduino framework for ESP32.

Basis: Original project by Karl Prosser (github.com/ReadyOS-C64/C64uRemote), ports for M5StickC Plus2 and M5Dial by Martin Oswald (1MHz.de); this Core version builds on those.

Hardware

Target platform

Property	Value
Board	M5Stack Core Basic / Gray / Fire
SoC	ESP32 (dual-core, 240 MHz)
Display	320 × 240, ILI9342C, via M5GFX
Controls	3 hardware buttons (A/B/C)
Memory	Core Basic without PSRAM (~250 kB usable heap)

Peripherals

Device	Connection	Details
Unit RFID2	Port A (Grove, red)	I²C, SDA = GPIO 21, SCL = GPIO 22, address 0x28, 100 kHz
microSD	internal slot	SPI, CS = GPIO 4, FAT32

Display and SD share the SPI bus. SD initialization first tries 20 MHz, and 4 MHz on failure.

Development environment

PlatformIO (recommended)

The configuration lives, version-controlled, in `platformio.ini`, which makes builds reproducible. Key settings:

Setting	Value	Reason
`board`	`m5stack-core-esp32`	Core Basic / Gray
`platform`	`espressif32`	Arduino framework
`board_build.partitions`	`huge_app.csv`	WiFi + SD + RFID exceed the standard app partition (1.25 MB)
`monitor_speed`	115200	serial output
`upload_speed`	460800	921600 is unstable on some USB-serial chips under macOS

Libraries (`lib_deps`):

- `m5stack/M5Unified` (≥ 0.2.8)
- `m5stack/M5GFX` (≥ 0.2.11)
- `bblanchon/ArduinoJson` (^6.21.5) — deliberately version 6, since version 7 no longer has `DynamicJsonDocument`
- `MFRC522_I2C` by kkloesener (Git) — I²C driver for the WS1850S

A second environment `m5stack-fire` additionally sets `-DBOARD_HAS_PSRAM`.

Commands:

```
pio run                 # compile
pio run -t upload       # flash
pio device monitor      # serial output
pio run -e m5stack-fire -t upload
```

Arduino IDE 2.x (alternative)

Rename `main.cpp` to `C64uRemote.ino`, install the libraries manually and choose **Huge APP** under *Tools → Partition Scheme*.

Configuration (build_env.h)

`build_env.h` only supplies the **initial values** now. As soon as a network configuration is present in NVS, the file is ignored. Copy the template `build_env.h.example` to `build_env.h` and fill it in:

```
#define C64U_WIFI_SSID       "MyWiFi"
#define C64U_WIFI_PASSWORD   "MyPassword"
#define C64U_TARGET_HOST     "192.168.0.64"
#define C64U_TARGET_PASSWORD ""       // only if set on the C64
```

If the file is missing, the project compiles with empty defaults; the Core then shows *SETUP > WIFI* at startup and can be set up on the device.

Wi-Fi subsystem

Runtime configuration instead of compile time

SSID, password, c64u address and c64u password live in NVS at runtime and can be changed through *SETUP → WiFi*. Up to `kWifiProfileMax` (= 4) profiles are kept:

```
struct WifiProfile { String ssid; String pass; };
WifiProfile gWifiProfiles[kWifiProfileMax];
size_t      gWifiCount;
size_t      gWifiTry;      // profile for the next connection attempt
```

`beginWiFi()` takes `gWifiProfiles[gWifiTry]` and then advances the index by one. If a connection fails, `serviceWiFi()` tries the next profile after `kWiFiRetryMs` – across several rounds the attempt therefore walks through all known networks. At start-up `wifiPickBestProfile()` scans the area once and begins with the strongest known network.

`wifiAddProfile()` sorts a new network in at the front; a network that is already known merely gets a new password and moves to the front as well. The oldest profile drops off the back if need be.

Screens and operation

Five screens are added. They are operated like every other list page: **A** and **C** page through, **B** triggers, **B long** goes back.

Screen	Purpose
`WifiMenu`	Submenu with the eight entries
`WifiScan`	List of the networks found
`WifiCard`	Present a card, read the password
`WifiPortal`	Access point running; shows SSID, password, IP and time left
`WifiSaved`	Stored networks: connect, delete or write to a card

The settings list now has an `enum SettingsId` with `kSetWifi` as the last action. `kSetLastAction` separates actions from switches, and a `static_assert` keeps the names and the labels in step.

NVS layout

Namespace `c64unet`, separate from the interaction settings in `c64uremote`:

Key	Contents
`wn`	number of profiles (0…4)
`s0…s3`	SSID
`p0…p3`	password
`host`	address of the c64u
`hpass`	password of the c64u

Factory Reset only touches `c64uremote`; the network configuration survives and is discarded exclusively through *WiFi → Delete all*.

Card format

Wi-Fi cards use the same scheme as Wi-Fi QR codes, stored as an ordinary NDEF text record:

`WIFI:S:<ssid>;T:WPA;P:<password>;;`

`parseWifiText()` evaluates the `S` and `P` fields, accepts backslash-escaped special characters and additionally understands the short form `WIFI:<ssid>;<password>`. `wifiCardText()` is the counterpart and is used by *WiFi → To NFC card*.

On the setup page (`ScreenMode::WifiCard`) a card text without the `WIFI:` prefix counts as a plain password for the network picked beforehand.

Outside the setup, `processCard()` recognises a complete `WIFI:` card, stores the network and connects right away. `gWifiTry` is pointed at that profile on purpose and the code waits up to `kWifiCardConnectMs` (8 s) for `WL_CONNECTED`; after that `serviceWiFi()` takes over again. If the network is already connected the function bails out with *ALREADY CONNECTED* – that prevents an endless loop when the card stays on the reader and the background poll spots it again.

/wifi.txt on the SD card

`loadWifiFromSd()` reads a simple key/value format. Every new `ssid` line begins an entry; `#` and `;` introduce comments. `host` and `hostpass` are recognised as well.

```
ssid = MyWiFi
pass = secret

host     = 192.168.0.64
hostpass =
```

The file is read at start-up (only while `gWifiCount == 0`) and on demand through *WiFi → Load from SD*.

`saveWifiToSd()` is the counterpart: it writes all profiles together with `host` and `hostpass` back to `/wifi.txt` with a comment header. An existing file is moved to `/wifi.bak` with `SD.rename()` beforehand; if that fails it is deleted. The function returns the number of networks written and puts a plain-text hint into `errorOut` on failure.

Setup portal

`startPortal()` switches to `WIFI_AP`, opens an access point on fixed channel 1 (`kPortalSsid` / `kPortalPass`), starts a `DNSServer` as a captive-portal redirect and a `WebServer` with two routes (`/` and `/save`). The station part is deliberately shut down: if it stayed active it would keep looking for the stored network in the background, change the radio channel while doing so and drop clients that had joined.

`servicePortal()` runs on every loop pass, holds the idle clock while a client is connected, and ends the portal after `kPortalIdleMs` (5 min) or `kPortalCloseMs` after a successful save. `stopPortal()` restores `WIFI_STA` and reconnects immediately.

Both classes come with the Arduino ESP32 core (`WebServer.h`, `DNSServer.h`); no extra libraries are needed.

Software architecture

State model

The entire runtime state lives in the global `struct AppState app`. The display follows a screen enum:

```
enum class ScreenMode {
  Home, CpuMenu, Status, Settings, SdBrowser,
  RfidRun, RfidWrite, RfidInfo, RfidDump, RfidRestore, Busy
};
```

The main loop `loop()` runs at ~30 fps (`kFrameMs = 33`):

1. `M5.update()` — update buttons and timers
2. `serviceWiFi()` / `refreshConnectionStatus()` — maintain the network
3. `handleButtons()` — evaluate button input
4. `serviceRfid()` — RFID polling (every 250 ms, only on RFID screens)
5. `updateHomeDemo()` — animation sequencing
6. `render()` — update the display

Rendering without PSRAM

The Core Basic has no PSRAM; three full-screen buffers (3 × 320 × 240 × 2 bytes ≈ 460 kB) do not fit in the heap. The StickC version uses such buffers – this port does not.

Instead, drawing happens **line by line directly to the display**. The logo lives in flash (`1MHz_logo_rgb565.h`, 240 × 135, RGB565) and is read via `pgm_read_word`. Additional RAM required: one line buffer (`rowBuf`, 640 bytes) and two scaling tables (`logoXMap`, `logoYMap`, together ~1 kB).

The effect computation (`drawDistortedRows`, `drawRotoZoom`, `drawRipple`, `drawRasterBars`) scales via `FX Detail`: with “Half” only every second pixel and every second line is computed and doubled on output (four times less compute). Menus are redrawn only on change (`screenDirty`, `barDirty`) so nothing flickers.

Battery indicator

The level comes from M5Unified: `M5.Power.getBatteryLevel()` (0-100, negative = no battery), `M5.Power.isCharging()` and `M5.Power.getVBUSVoltage()`. It is read at most every `kBattPollMs` (5 s); blinking and swapping run on the remembered value and cost no further I2C traffic.

The Core has an **IP5306**. There `getBatteryLevel()` only returns 0, 25, 50, 75 or 100, and it does not know `getVBUSVoltage()` (returns −1) - "on power" here therefore simply means "currently charging".

Drawing happens in three places:

- `drawBatteryLine()` replaces the line below the bar. The base line stays `kColLine`, the filled part is two pixels tall. At 0 % it would be too narrow to see, hence `if (width < 4) width = 4;`.
- `drawBatterySymbol()` draws the frame (32 x 12 px) and the terminal and puts the percentage in the middle. The frame is deliberately not filled - the line does that, and the number stays readable.
- `drawChargeBolt()` puts a 5 x 7 pixel bolt to the left of it, drawn row by row from a small table instead of from triangles; at this size the shape would not hold otherwise.

The colour for all three comes from `batteryColor()`, the thresholds are `kBattGreen`, `kBattYellow` and `kBattBlinkAt` at the top of the file.

`drawStatusBar()` used to run at most once per second - too rarely for visible blinking. On the lowest step `render()` therefore shortens the interval to `kBattBlinkMs` (500 ms).

ReST connection to the C64 Ultimate

All commands go through the HTTP ReST API of the Ultimate firmware (from 3.11). Base URL: `http://<host>/v1/…`. If a network password is set, it is sent in the `X-Password` header.

Endpoints used

- **Version / reachability** — `GET /v1/version`
- **Reset** — `PUT /v1/machine:reset`
- **Reboot** — `PUT /v1/machine:reboot`
- **Power off** — `PUT /v1/machine:poweroff`
- **Ultimate menu** — `PUT /v1/machine:menu_button`
- **Write memory** — `PUT/POST /v1/machine:writemem?address=…`
- **Read memory** — `GET /v1/machine:readmem?address=…&length=…`
- **Config categories** — `GET /v1/configs`
- **Read/set config** — `GET/PUT /v1/configs/<category>/<item>`
- **Start PRG** — `POST /v1/runners:run_prg`
- **Start CRT** — `POST /v1/runners:run_crt`
- **Play SID/MOD** — `POST /v1/runners:sidplay` / `:modplay`
- **Mount disk** — `POST /v1/drives/<a|b>:mount?type=…&mode=…`
- **Drive on** — `PUT /v1/drives/<a|b>:on`

`sendApiRequest()` wraps GET/PUT over the `HTTPClient` (3 s timeout) and parses the JSON response with ArduinoJson (`errors` array).

Refused connections

The HTTP server of the Ultimate firmware accepts only one connection at a time and refuses further ones with a TCP RST; `HTTPClient` reports this as *connection refused*. It was observed with only a single device on the network as well - so it happens sporadically and is neither a radio nor an address problem.

The actual call therefore moved into `sendApiRequestOnce()`. `sendApiRequest()` is only a wrapper around it: if the transport fails (`httpCode <= 0`), a second attempt follows after `kApiRetryDelayMs` (250 ms). Retrying happens **only** on transport errors - nothing reached the c64u then, so a

command cannot be doubled. HTTP error statuses (4xx, 5xx) are passed through unchanged, and the streaming upload in `uploadFile()` has its own path and stays untouched.

On top of that `refreshConnectionStatus()` saves a request: without a stored password the second query would be byte-identical to the first, because the `X-Password` header is only set when there is one. That halves the base load on the c64u.

Reconnecting with several networks

`beginWiFi()` moves on to the next stored profile after every attempt. Without a countermeasure that means: with two networks stored of which only one is reachable, every other reconnect after a dropout hits the dead one and costs a full retry cycle (`kWiFiRetryMs`, 10 s).

`serviceWiFi()` therefore remembers the profile the connection came up on, via `wifiProfileIndex(WiFi.SSID())`, and makes it the next attempt; `gWifiNoted` keeps this to once per connection. After a dropout the first attempt goes back to the working network, and the dead one is only tried if the good one is really gone.

CPU speed

The path to the CPU-speed item is not fixed but discovered: first in “U64 Specific Settings”, otherwise across all categories (`resolveCpuPath`). The available values come from the item’s `values` array; a fixed list serves as a fallback (`setFallbackCpuChoices`).

Joystick ports

There is no `machine:` command for swapping the joystick ports. The mapping is a configuration item - in testing *Joystick Swapper* in the category *U64 Specific Settings*, with the values `Normal`, `Swapped`, `WASD Port 2` and `WASD Port 1`. It is therefore set through `/v1/configs/<category>/<item>?value=…`, exactly like the clock speed.

As with the clock, `resolveJoyPath()` looks in *U64 Specific Settings* first and otherwise walks all categories until an item name contains “Joystick”, so a later firmware renaming it goes unnoticed. `refreshJoyChoices()` reads `values` and `current`.

`joyTokenFromValue()` and `joyValueFromToken()` convert between the device value and the card token (`WASD Port 1 <-> WASD1`), so a card needs neither a blank nor a firmware-specific spelling. `toggleJoystickSwap()` toggles between `Normal` and `Swapped` and returns to `Normal` from a WASD mode; `cycleJoystickValue()` walks through every reported value in the setup list.

Disk images and autostart

When mounting, the target drive is determined (`resolveTargetDrive`): with “Auto” the Core searches via `GET /v1/drives` for the drive with `bus_id: 8`. After mounting comes `:on`, then depending on *Disk Action* a reset and an autostart.

The autostart types via DMA into the C64 keyboard buffer (`writemem` at `$0277` ff., count register `$C6`). It uses the abbreviated BASIC commands `lO"*",8,1` and `rU`, so the line fits into the ten bytes of the buffer.

The wait times are **not fixed** but governed by `$CC` (BLNSW, cursor blink): `waitCursorBlinking()` polls `readmem` and detects from the blinking when BASIC is ready and when loading has finished. Load timeout: 3 minutes.

Streaming upload

Since large `.d64` files (up to ~800 kB for `.d81`) do not fit in RAM, `uploadFile()` sends the file in blocks (1 kB) directly from the SD stream into a `WiFiClient` socket. Multipart boundaries and headers are built manually; the `type/mode` arguments go in the URL, not as a form field (otherwise the firmware interprets every multipart part as a file).

NFC subsystem

Polling strategy

`serviceRfid()` runs in two modes:

1. **On the RFID screens** (`RfidRun`, `RfidWrite`, `RfidInfo`, `RfidDump`, `RfidRestore`) a full `cardPresent()` poll happens every 250 ms.
2. **On the main screen** a quick probe with `cardPresentQuick()` runs at the interval configured under *Auto-NFC*. When it finds a card, the code sets `autoRfidActive`, switches to `RfidRun` through `setScreen()`, draws one `render()` and then calls the same handling the reading screen uses.

The actual card handling lives in `processCard()`. Both paths call it, so there is only one implementation.

Why a dedicated probe

With no card present, the MFRC522 waits after the REQA command until its internal timer expires. `PCD_Init()` sets that to 0x03E8 = 1000 steps of 25 µs, i.e. 25 ms. The main loop stalls for exactly that long, which would show up as a stutter while an animation is running.

A card answers far more quickly: the frame delay time at 106 kbit/s is roughly 86 µs. `cardPresentQuick()` therefore shortens the window to about 2 ms for the probe only and restores it immediately afterwards — before `PICC_ReadCardSerial()`. Selection, authentication and every write operation still run with the full window.

```
void setRfidTimerReload(uint16_t ticks) {          // 1 tick = 25 µs
  rfid.PCD_WriteRegister(MFRC522_I2C::TReloadRegH, ticks >> 8);
  rfid.PCD_WriteRegister(MFRC522_I2C::TReloadRegL, ticks & 0xFF);
}
```

The original value is not hard-coded but read from `TReloadRegH/L` in `initRfid()` and kept in `gRfidTimerReload`. Should a future library version change the default, the behaviour stays correct.

Cost. A probe that finds nothing consists of a handful of I²C register accesses plus the roughly 2 ms wait, some 5 ms in total. At the default interval of 700 ms that is a background load below one per cent, not enough to miss a 33 ms frame.

Auto-NFC	Interval	approximate load
Off	–	0 %
1.5s	1500 ms	~0.3 %
0.7s	700 ms	~0.7 %
0.3s	300 ms	~1.7 %

Returning to the main screen

A reading screen opened automatically falls back after `kAutoRfidHoldMs` (20 s), leaving time to present further cards. The flag `app.autoRfidActive` distinguishes it from a screen the user selected deliberately, which stays put. Any button press and any manual navigation clear the flag.

Card types

Family	Detection	Access
MIFARE Classic 1K/4K/Mini	SAK	16-byte blocks, authentication required
NTAG213/215/216, Ultralight	SAK 0x00	4-byte pages, no key

`cardKind()` distinguishes by the SAK byte. The exact NTAG type comes from the `GET_VERSION` command (0x60), or as a fallback from the Capability Container (page 3).

Command cards

If a card carries the prefix `CMD:` instead of a file path, its content is executed as a command for the c64u. Neither an SD card nor a file is needed.

```
CMD:RESET
CMD:REBOOT
CMD:MENU
CMD:POWEROFF=0      power off immediately
CMD:POWEROFF=8      ask first, 8 s confirmation window
CMD:POWEROFF        ask first, using the device setting "NFC-Cmd PowOff"
CMD:CPU=10          set the CPU to 10 MHz
CMD:JOY             toggle the joystick ports (Normal <-> Swapped)
CMD:JOY=SWAPPED     set the ports fixed; also NORMAL, WASD1, WASD2
```

`parseCardCommand()` takes the text apart: check the prefix, split off an optional argument after `=`, compare the keyword in upper case. Whitespace and letter case do not matter. The content stays a plain NDEF text record, so any NFC app can read and write such a card.

In its reading branch `processCard()` checks for a command first and calls `runCardCommand()`. If a card carries the prefix but no known keyword, the device reports *BEFEHL UNBEKANNT* instead of turning it into a file path.

PowerOff with confirmation

The waiting time is an argument **on the card**, not in the device; `cardPowerOffSeconds()` returns it. Without an argument the setting *NFC-Cmd PowOff* applies (3/5/8/15 s), `0` means “no prompt”.

The sequence uses three fields in `AppState`:

```
bool     cardPowerOffPending;
String   cardPowerOffUid;      // only the same card confirms
uint32_t cardPowerOffUntilMs;
```

Confirmation happens by presenting the same card again or by pressing the button. A different card cancels, and so does letting the window expire - both clear the flag without triggering anything. Binding to the UID prevents an unrelated card placed nearby from powering the machine down.

Writing cards

`cmdListAt()` builds the selection list: five fixed commands, then every CPU step currently held in `app.cpuDisplayOptions` (loaded from the c64u, otherwise the built-in fallback list). `cardCommandText()` turns the choice into the card text. The screen `ScreenMode::CmdPick` shows the list; the selection ends up in `app.pendingCardText`, which the writing screen prefers over `pathToCardText(app.pendingPath)`.

Data format (TeensyROM/Zaparoo-compatible)

The card holds a single **NDEF record of type Text** (Well Known, UTF-8). The content is the path to the program file:

```
SD:OneLoad v5/Bubble Bobble.crt
```

Prefixes `SD:`, `USB:`, `TR:` are accepted (the latter two are searched on the SD). A `?` as the filename, or a directory path, launches a random file from the folder. Maximum text length: 246 characters.

Storage:

- NTAG/Ultralight: NDEF TLV from page 4, pages 0–3 remain untouched.
- MIFARE Classic: NDEF TLV in the data blocks from block 4, trailers are skipped. Authentication first with the NDEF key `D3F7D3F7D3F7`, otherwise the factory key `FFFFFFFFFFFF`.

NDEF parser: tolerance

`parseNdefText()` reads the text robustly. Important edge cases:

- **Wrong payload length:** the TeensyROM writes a constant `0x10` into the length byte, even though the TLV length is correct. For the last record (ME flag) the TLV length therefore takes precedence, otherwise the path would be truncated.
- Filler bytes (`0x00`) and the TLV terminator (`0xFE`) end the text.
- Multiple slashes (`SD://folder`) are collapsed.
- Leading Lock-Control TLVs and 3-byte lengths are skipped.

For writing, `buildNdefText()` produces a clean short record. The former raw format `C64UPATH` is still recognized when reading.

NFC-Info

`collectCardInfo()` fills two pages (switchable on the device with A/C):

- **Page 1:** UID, SAK, type, memory size, version/manufacturer, content, path, filename and a check against the SD card.
- **Page 2:** hex dump of pages 0–15, lock bytes, decoded Capability Container, password protection from the config page and the NFC read counter.

Commands that not every card supports (`GET_VERSION`, config page, `READ_CNT`) deliberately run last, since an unsupported command deselects the card. After a failed `GET_VERSION` the card is made responsive again via `reselectCard()` (WUPA + Select). A read card is read only once per UID and then buffered, so the display is not rebuilt on every poll.

Copying (dump/restore)

`dumpCardToSd()` writes the card content as a text file to `/NFC-DUMPS/<uid>.nfc`. For MIFARE Classic a **key dictionary** (13 common keys) is tried per sector; the key found is noted as a comment and inserted into the trailer line (since key A always reads back as `00…`).

`restoreDumpToCard()` writes back the data blocks and – with valid access bits – the sector trailers as well. Not written are:

- **Block 0** (UID) — write-protected on normal cards.
- **Key A** (trailer bytes 0–5) — fundamentally not readable.
- Trailers with **invalid access bits** — `validAccessBits()` checks the complement encoding to prevent permanently locking the sector.

Data blocks are always written before the corresponding trailer, so access within the sector is not lost.

Control logic

Button handling

`handleButtons()` evaluates all three buttons per iteration. In addition to short and long presses there are four freely assignable actions (`ShortcutAction`: None, Reset, Reboot, UltiMenu, PowerOff):

- **B long** (released without a second key)
- **C long**
- **B long, then C** (sequence or chord)
- **B long, then A**

The sequence enables one-handed use: after releasing B a time window opens (`Kombi Zeit`, 0.5–3.0 s) in which A or C triggers the combination. While B is held and during the wait window, A and C are decoupled from their normal functions.

In the SD browser the long presses are mapped differently (A long = folder back, B long = home, C hold = auto-scroll) so that fast scrolling does not collide with the shortcuts.

PowerOff safeguard

Two separate confirmation paths:

- **POWER tile:** a second press of B within `PowerOff Zeit`.
- **Key shortcut:** depending on `PowerOff Kombi` either directly or with an A confirmation. The check runs before the B evaluation, so the A key that triggers "B long + A" does not immediately confirm its own prompt.

Persistence

Settings live in NVS (`Preferences`, namespace `c64uremote`). Time values are stored in tenths of a second and clamped to the valid range on load. `loadDefaultSettings()` restores the factory defaults.

The Wi-Fi credentials live in a namespace of **their own**, `c64unet` (see the *Wi-Fi subsystem* chapter), so they survive a *Factory Reset*. `build_env.h` only supplies the initial values as long as nothing has been stored there.

Project structure

```
M5Core_C64uRemote/
├── platformio.ini            board, libraries, partition, upload
├── README.md
├── LICENSE                   MIT - Karl Prosser, Martin Oswald
├── wifi.txt.example          template for /wifi.txt on the SD card
├── .vscode/                  recommended extensions, editor settings
├── docs-src/                 markdown sources of the manuals
├── doc/                      finished manuals (PDF)
├── src/                      German edition
│   ├── main.cpp              entire firmware
│   ├── build_env.h           credentials (do not version)
│   ├── build_env.h.example   template
│   └── 1MHz_logo_rgb565.h    logo as an RGB565 array in flash
└── src-en/
    └── main.cpp              English edition, identical code
```

Troubleshooting

The serial monitor (115200 baud) prints heap, RFID and SD status on startup. When reading a card it logs the text read, for rejected files the path and extension, and for a disk mount the full URL. For on-device NFC analysis use NFC-Info page 2.

Typical messages:

Message	Cause
`SET build_env.h`	credentials missing
`NO WIFI`	no Wi-Fi connection
`AUTH?` (status bar)	network password set on the C64, not stored here
`TYP UNBEKANNT: …`	file extension not supported by the Ultimate
`KARTE OHNE C64U-PFAD`	card contains no recognizable path
`LADEN DAUERT ZU LANGE`	disk autostart after 3 min without READY

Sources

- ReST API of the Ultimate firmware: 1541u-documentation.readthedocs.io/en/latest/api/api_calls.html
- TeensyROM NFC Loader (card format): github.com/SensoriumEmbedded/TeensyROM/blob/main/docs/

- Reference implementation ultimate64 (Rust): github.com/mlund/ultimate64
- MFRC522_I2C library: github.com/kkloesener/MFRC522_I2C
- Original project C64uRemote: github.com/ReadyOS-C64/C64uRemote

License

C64uRemote is released under the **MIT License**. The full text is in the file `LICENSE` in the project root.

It originates from **C64uRemote by Karl Prosser (@klumsy)**, https://github.com/ReadyOS-C64/C64uRemote, which he published under the MIT License. This version is a derivative work and is released under the same terms:

- Copyright (c) 2026 Karl Prosser – original project
- Copyright (c) 2026 Martin Oswald (@mad, https://1MHz.de) – port and extensions

The MIT License allows you to use, modify and redistribute the software, including commercially. The only condition: **the copyright notice and the license text must be kept** and included with every copy. There is no warranty and no liability.

The libraries used carry their own licenses: M5Unified and M5GFX (MIT, © M5Stack), ArduinoJson (MIT, © Benoit Blanchon) and MFRC522_I2C (https://github.com/kkloesener/MFRC522_I2C). PlatformIO fetches them at build time.

How this was made

The port, the extensions and these manuals were written with the help of Claude (Anthropic). Concept, idea, hardware decisions and every test on real devices: Martin Oswald (@mad).